PRACTICAL NUMBER: 6

AIM: Program based on tries:

Create a text file containing keys{“and”, “bat”, “ ball”,

“book”,“cot”, “cotton”, “internet”, “interview”, “joy”,

“job”, “King”, “Lion”, “man”, “mango”, “manage”}

separated by tab and then read those keys and insert them

in trie and search the key which is accepted as input from

user.Read the keys from char array which is already given and

search the keys which are accepted as input from user. Read the keys from char array and search those keys

which are given in text file***PROGRAM*:**

```
//sheikh wasimuddin 60 23071044 cseiot//
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
typedef struct TrieNode {
   struct TrieNode *children[26];
   bool is_end_of_word;
} TrieNode;
TrieNode* createNode() {
   TrieNode *node = (TrieNode *)malloc(sizeof(TrieNode));
   node->is_end_of_word = false;
```

```
    for (int i = 0; i < 26; i++)
        node->children[i] = NULL;
    return node;
}
void insert(TrieNode *root, const char *word) {
    TrieNode *node = root;
    for (int i = 0; word[i] != '\0'; i++) {
        int index = word[i] - 'a';
        if (node->children[index] == NULL) {
            node->children[index] = createNode();
        }
        node = node->children[index];
    }
    node->is_end_of_word = true;
}
bool search(TrieNode *root, const char *word) {
    TrieNode *node = root;
    for (int i = 0; word[i] != '\0'; i++) {
        int index = word[i] - 'a';
        if (node->children[index] == NULL) {
            return false;
        }
        node = node->children[index];
    }
```

```
    return node->is_end_of_word;
}
void createWordFile(const char *filename) {
    const char *words =
"and\tbat\tball\tcot\tcotton\tinternet\tinterview\tjoy\tjob\tking\tlio
n\tman\tmango\tmanage";
    FILE *file = fopen(filename, "w");
    if (file) {
        fprintf(file, "%s", words);
        fclose(file);
    } else {
        perror("Failed to create textfile");
    }
}
void loadWordsFromFile(TrieNode *root, const char *filename) {
    FILE *file = fopen(filename, "r");
    if (file) {
        char word[100];
        while (fscanf(file, "%s", word) != EOF) {
            insert(root, word);
        }
        fclose(file);
    } else {
        perror("Failed to open textfile");
```

```
    }
}
int main() {
    const char *filename = "words.txt";
    createWordFile(filename);
    TrieNode *trie = createNode();
    loadWordsFromFile(trie, filename);
    char search_word[100];
    printf("Enter a word to search in the trie: ");
    scanf("%s", search_word);
    if (search(trie, search_word)) {
        printf("'%s' is found in the Trie.\n", search_word);
    } else {
        printf("'%s' is not found in the Trie.\n", search_word);
    }
    return 0;
}
```

OUTPUT:

main.c | words.txt

```c
//sheikh wasimuddin 60 23071044 cseiot//
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>



typedef struct TrieNode {
    struct TrieNode *children[26];
    bool is_end_of_word;
} TrieNode;


TrieNode* createNode() {
    TrieNode *node = (TrieNode *)malloc(sizeof(TrieNode));
    node->is_end_of_word = false;
    for (int i = 0; i < 26; i++)
        node->children[i] = NULL;
    return node;
}


void insert(TrieNode *root, const char *word) {
    TrieNode *node = root;
    for (int i = 0; word[i] != '\0'; i++) {
        int index = word[i] - 'a';
        if (node->children[index] == NULL) {
            node->children[index] = createNode();
        }
        node = node->children[index];
    }
    node->is_end_of_word = true;
}
```

```
Enter a word to search in the trie: king
'king' is found in the Trie.


...Program finished with exit code 0
Press ENTER to exit console.
```

CONCLUSION: Program based on tries was performed successfully.